FirmGen Hands-On Lab: From PRD to Working Firmware

A 90 minute participant guide for the FirmGen Hackathon

FirmGen v0.4.1

Abstract

This lab introduces FirmGen as a complete firmware development workflow rather than a one time code generator. In 90 minutes you set up FirmGen (if not already done) and take one small PRD, an **ESP32 BLE broadcast device**, all the way from specification to firmware running on your board, with reviewed requirements, acceptance criteria, unit tests, design documentation and traceability along the way.

Everyone uses the same PRD, so you can focus on the workflow, how the tool is used at each stage, and what each stage produces. The full PRD is in Appendix A. While FirmGen implements the firmware, you use the waiting time to explore the rest of the workspace. The Reference section explains each FirmGen capability in more detail so you can work through problems on your own.

Learning objectives

By the end of the session, you should be able to:

- Install the application and start it.
- Create a FirmGen project with the correct platform, board and toolchain, and know what the optional knowledge inputs are for.
- Upload a PRD, generate requirements, tell Explicit from Inferred, and freeze a reviewed set.
- Generate task lists and acceptance criteria, choose an execution path, and start implementation.
- Validate the firmware on the board against the PRD acceptance criteria.
- Generate unit tests and a design document, follow a requirement through the traceability report, and push your work with Git.
- Read the deploy evidence and tell the difference between a command that completed and firmware that actually works on the target.

How to use this guide

Follow Steps 1 to 9 in order during the session. Step 1 can be skipped if FirmGen and its prerequisites are already installed on your machine. After the lab, the "Continuing After the Lab" section, the Reference section and the Quick Recovery Guide help you use FirmGen on your own projects.

> **Important:** project names shown in screenshots are only examples, and screens on your machine may differ slightly.

FirmGen at a Glance

Capability	What it does for you	Where in this guide
Requirements-driven workflow	Turn a PRD into reviewed requirements, task lists and acceptance criteria before any code is written.	Steps 3 to 6
Three execution paths	Direct Prompt, Auto Mode or Category-wise Manual Review, depending on how much review you want.	Step 7

Capability	What it does for you	Where in this guide
Knowledge inputs	Coding guideline, board schematic, datasheets and other sources that guide generation.	Step 2
Artifacts	Requirement-linked unit tests that FirmGen runs for you, and a generated design document.	Step 9
Traceability	Follow any requirement to the code and tests that implement it, and back again.	Step 9, R1
Git	Push and pull, commit history and file-level diffs inside the workspace.	Steps 7 and 9, R3
Direct prompting	Describe a change in the chat; FirmGen plans it, edits the project, and runs build, flash and monitor.	After the lab
Live Intelligence	Breakpoints and runtime monitoring for telemetry-enabled firmware.	R2

Supported platforms

FirmGen v0.4.1 supports **ESP32** (with **ESP-IDF v5.5**) and **STM32**. Both are available in the platform selector on the project setup page; for STM32 you also choose the chip family there. This lab uses ESP32. Other platforms you may see in the selector are not part of this hackathon.

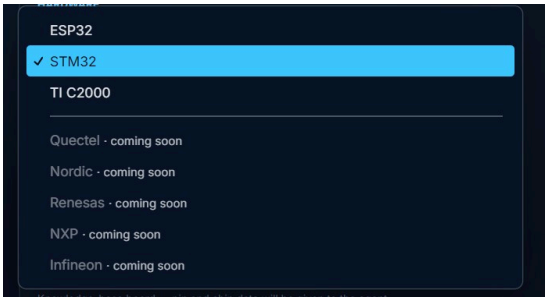


Figure 1. The platform selector on the project setup page.

What You Need and the 90 Minute Plan

What you need

Item	Why
ESP32 development board and USB cable	The PRD uses BLE. The STM32 boards in the kit do not have BLE, so if your table only has an STM32 board, ask a facilitator for an ESP32.
Laptop with FirmGen	Prerequisites and the application already installed.
The PRD file **ESP32_BLE_FirmGen_Simple_PRD.pdf**	Provided at the event. The same text is in Appendix A.
A phone with a BLE scanner app	For the final acceptance check: seeing your device advertise.

No wiring is needed for this lab. The sensors and modules in the kit are for your hackathon builds afterwards.

The 90 minute plan

Time	Step	Outcome
0 to 10 min	Step 1. Install the prerequisites and FirmGen Application(skip if already done).	Able to Launch FirmGen
10 to 15 min	Step 2. Create the BLE project.	A valid FirmGen workspace anchored to your ESP32 board.
15 to 18 min	Step 3. Upload the PRD and start requirement generation.	PRD saved; requirements generating.
18 to 24 min	Step 4. Tour the workspace while requirements generate.	You know where to find everything you will use next.
24 to 31 min	Step 5. Review and freeze the requirements.	A reviewed, frozen requirement baseline.
31 to 35 min	Step 6. Generate task lists and derive criteria.	A visible plan and acceptance criteria per requirement.
35 to 80 min	Step 7. Start implementation in Auto Mode, then explore while it runs (Git, Topology, Files).	Firmware implemented, built and flashed.
80 to 85 min	Step 8. Validate on the board and with a BLE scanner.	All three PRD acceptance criteria checked.
85 to 90 min	Step 9. Artifacts, traceability and Git push.	Unit tests, design doc and traceability report; work pushed.

Keeping to time: implementation is the longest step, and its duration depends on your machine, build iterations and the board. If a step runs long, do not restart it: move on to what you can do in parallel (Step 4 during requirement generation, Git and exploring during implementation) and follow your facilitator's timing.

The workflow at a glance

PRD → Requirements (review + freeze) → Task lists → Criteria → Implementation → Build / Flash / Monitor → Artifacts → Traceability

Use this table as your map. After each stage, find the artifact before moving on.

Stage	What you do	What FirmGen produces	Where to find it
PRD	Upload the BLE PRD.	The PRD, kept as the original specification reference.	Project **sources/** folder
Requirements	Generate, review, freeze.	Categorized requirements with stable IDs, tagged Explicit or Inferred.	**Req.** tab; requirements table in sources/
Task lists	Plan all categories.	Requirement-linked implementation tasks per category.	Inside each category in the Req. tab
Criteria	Derive and review.	Acceptance criteria per requirement, with how each will be checked.	Req. tab; criteria file in sources/
Implementation	Start implementation.	Modular firmware source code.	**Files** tab; firmware folders

Stage	What you do	What FirmGen produces	Where to find it
Build / Flash / Monitor	Deploy to the board.	Firmware on the board and UART evidence.	Pipeline, Output and Monitor
Unit tests	Artifacts → Unit Tests.	A test per requirement, executed, plus a results report.	In the Artifacts tab
Design doc	Artifacts → Design Doc.	Design specification: wiring, HLD, LLD, state and sequence flows.	Design folder under the project docs
Traceability	Open the report.	Requirement → criteria → code → tests → status.	Traceability report in sources/
Versioning	Commit and push.	History and diffs of everything above.	Git view

Exact folder and file names may vary slightly by project.

Step 1: Install the Prerequisites (skip if already done)

FirmGen builds and flashes with tools installed on your machine. If you have already installed the prerequisites, confirming the required versions, and the tool launches successfully, skip to Step 2.

1. Download the **tool** from the CraftifAI developer portal as per your OS, refer to the prerequisites guide for installing all of the prerequisites from the given zipped folder, install FirmGen Application, and launch the tool.
2. Run it. It automatically installs the ESP32 toolchain, including **ESP-IDF v5.5**, which FirmGen supports.
3. If there is any error, please check the versions of your prerequisites carefully, and reinstall them.

> **Keep to ESP-IDF v5.5:** if you already have another ESP-IDF version on your machine, make sure FirmGen is using v5.5. A mismatched version is a common cause of build failures that look like problems in the generated code.

Step 2: Create the BLE Project

Open FirmGen. On the project setup screen, keep **Create new** selected and fill in the fields.

Field	What to enter	What to observe
Project name	For example **FirmGen_BLE**.	The display name clearly identifies the project.
Parent directory	Browse to a writable workspace folder.	You can see your generated files under this code.
Platform	**ESP32**.	The board list updates.
Board	Select your supplied ESP32 board and click **Verify**.	The board is recognized and the project is anchored to it.
Toolchain	Confirm the ESP-IDF v5.5 toolchain from Step 1 is selected.	FirmGen can initialize the toolchain.
Coding guideline	Optional. Try **BARR-C Embedded C Coding Standard** or **Defaults** to see the effect on the generated code.	The rule set shapes naming, APIs, modularity and error handling.
Board schematic, Sources	Leave empty. The PRD needs nothing extra.	Used for custom boards or extra reference material.

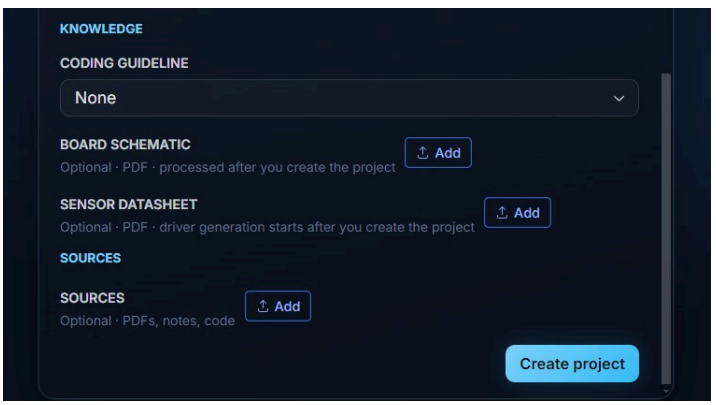


Figure 2. Knowledge inputs on the project setup screen. Your screen may show slightly different fields.

Click **Create project**. A valid project opens with no firmware topology and no plan yet; this is expected.

If the Create project stays disabled: the usual causes are an unverified board, an unresolved toolchain entry, or a parent directory that is not writable. Fix the flagged field rather than recreating the project.

Step 3: Upload the PRD and Generate Requirements

1. Select the small document control beside the chat composer. FirmGen pauses the normal chat and opens **Draft PRD**.
2. Choose **I have a document** and upload **ESP32_BLE_FirmGen_Simple_PRD.pdf**. It is saved into the project sources in seconds.
3. Read the saved PRD in the panel, then select **Generate requirements**. Progress appears in the review panel and the left sidebar. It usually takes a few minutes: go straight to Step 4 while it runs.

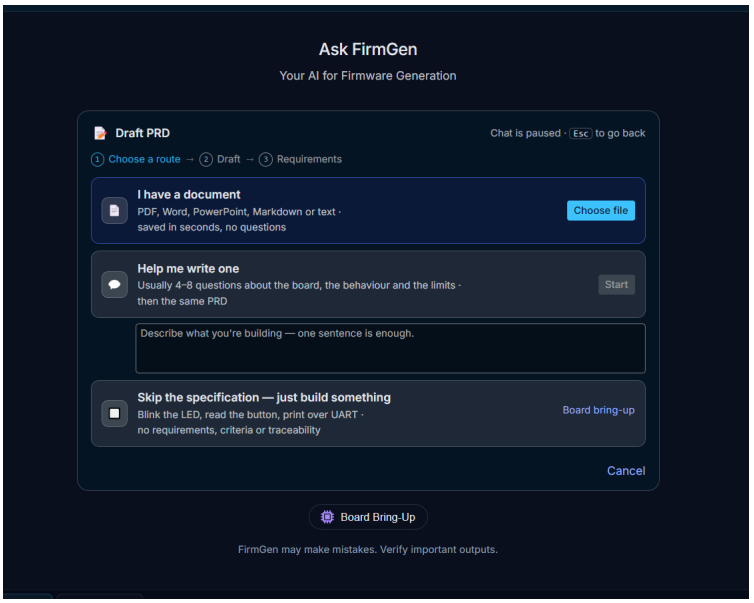


Figure 3. Draft PRD routes. Use "I have a document" for the BLE PRD.

The other two routes are **Help me write one** (a one-sentence description plus usually 4 to 8 questions about the board, behavior and limits) and **Skip the specification** (a quick board bring-up with no requirements, criteria or traceability). You will use them on your own projects.

Step 4: Tour the Workspace While Requirements Generate

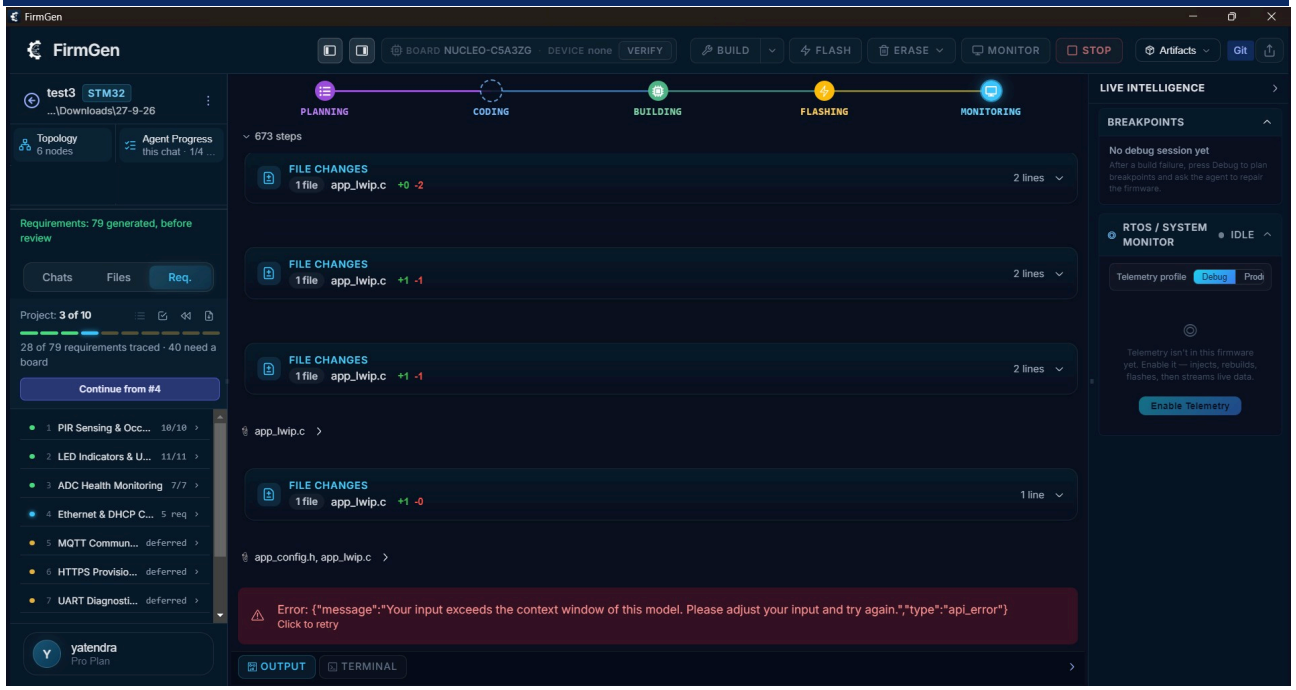


Figure 4. The FirmGen workspace during a long-running project. This example also shows a context-window error, which is covered in the Reference section.

Area	Purpose	When you use it in this lab
Project panel (top left)	Project name, platform badge, path, plus Topology and Agent Progress.	Confirm you are in the BLE project.
Background status lines	Progress of requirement generation and other background work.	Now: watch requirement generation.
Chats / Files / Req. tabs	Chat history, generated files, and the requirements control surface.	Req. from Step 5; Files in Step 7.
Prompt composer	Where you type prompts. The document control beside it opens Draft PRD. A context usage indicator shows how full the chat is.	Step 3; after the lab.
Progress pipeline	Planning, Coding, Building, Flashing, Monitoring, with file-change cards.	Step 7.
Top bar	Board and device with Verify; Build, Flash, Erase, Monitor, Stop; Artifacts; Git.	Steps 7 to 9.
Output / Terminal	Tool output and terminal.	If a build fails: read the first meaningful error.
Live Intelligence (right)	Breakpoints and RTOS / System Monitor.	Optional. See R2.

Before implementation starts: connect the ESP32 by USB and note which serial port it appears on. Monitor holds that port while it is running, so stop Monitor before FirmGen needs to flash.

Step 5: Review and Freeze the Requirements

Requirements are grouped into categories. Each is tagged **Explicit** (taken directly from the PRD) or **Inferred** (added by FirmGen to make the system implementable or verifiable, shown with its rationale).

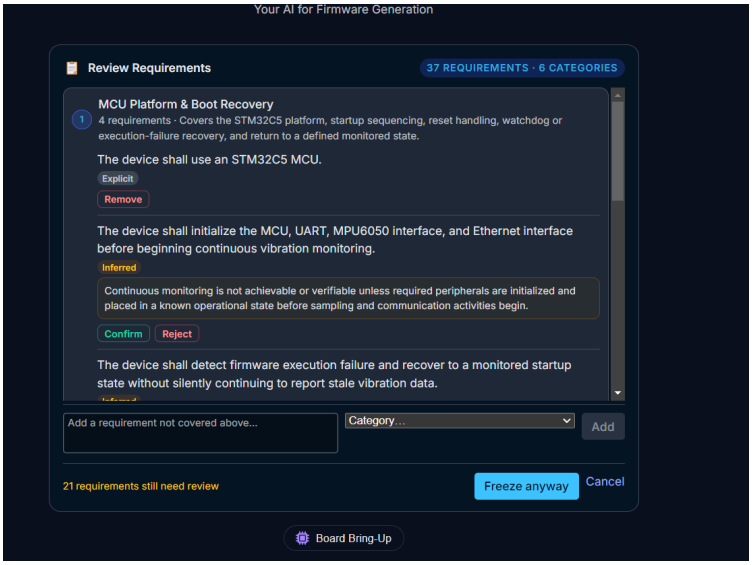


Figure 5. Review panel with categories, Explicit and Inferred tags, and review actions. The screenshot shows a different sample project.

Review action	Purpose
Confirm	Accept an inferred requirement and its rationale into the frozen specification.
Reject / Remove	Exclude an incorrect, unnecessary or out-of-scope requirement.
Add	Write a missing requirement and assign it to a category.
Freeze	Commit the reviewed set so planning and implementation use a stable baseline.

Check against the PRD:

- Each of the six numbered requirements, the three UART log messages and the 115200 baud setting appears as an Explicit requirement.
- The name format is captured exactly: FirmGen_ followed by the last 2 hex digits of the MAC address.
- Inferred requirements stay within scope. The PRD only asks for advertising, so anything that adds connections, pairing or data services goes beyond it: reject it. Extra scope also makes implementation take longer.
- Read every inferred requirement before you select **Freeze**. The panel lets you Freeze anyway, but this is where wrong assumptions enter.

Step 6: Generate Task Lists and Derive Criteria

Once frozen, the categories appear in the **Req.** tab, the project control surface: category count, requirement count, traceability progress, planning status and the implementation entry point.

1. Select **Task lists** to plan every category. Open one category and read its tasks: each names the requirement(s) it satisfies.
2. Select **Criteria** to derive acceptance criteria: how FirmGen will decide each requirement is implemented correctly, and whether it is checked on the host, on the target, by inspection, or with instruments.
3. Look at how each criterion will be checked. For example, building FirmGen_XX from a MAC address is logic that can be checked off the board, while "visible from a BLE scanner" can only be checked on the board.

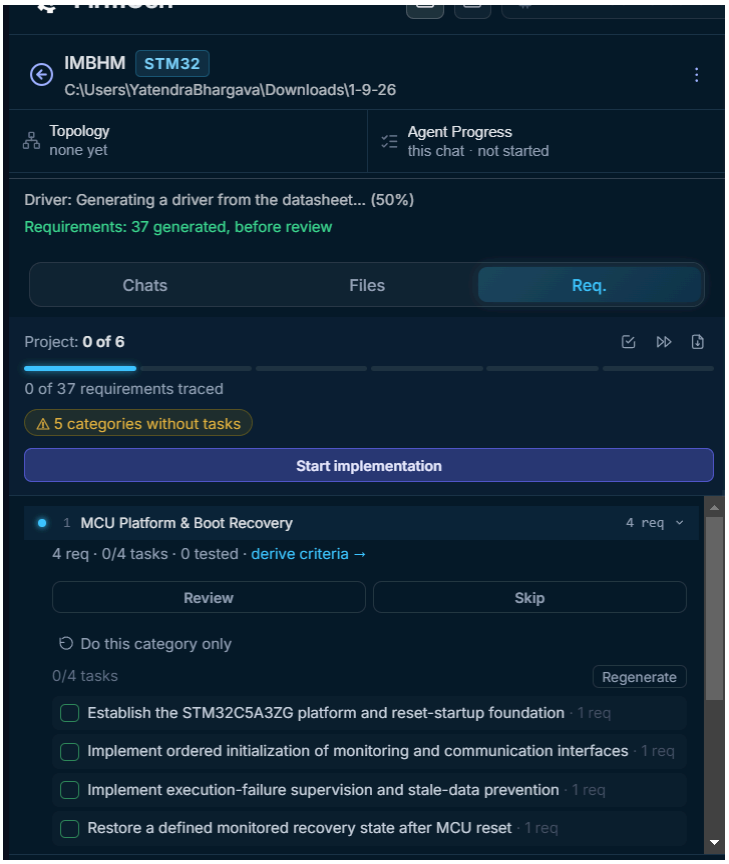


Figure 6. An expanded category with requirement-linked tasks, derive criteria, Review, Skip, Do this category only, and Regenerate.

If planning takes longer than expected: you can start implementation anyway. A category without a task list is still implementable; Auto Mode plans it as it goes.

Step 7: Implement in Auto Mode, and Explore While It Runs

7-1. Choose the execution path and start

Path	Behavior	Indicative duration
Auto Mode	Skips category review pauses and continues through implementation; plans missing categories and performs runtime testing where required.	~45 min or more
Category-wise Manual Review	Runs every category with review and execution checkpoints. You inspect tasks and code, and accept, reject or regenerate.	~1 to 3 hr or more

For this lab, use **Auto Mode**: it runs the full requirements-driven flow, including runtime testing, without review pauses, so it fits the session. Then select **Start implementation**. Progress is shown per category in the Req. panel and in the pipeline. Category-wise Manual Review gives you the most control and is worth using on your own projects when you have more time.

Do not interrupt the run: avoid pressing Build, Flash or Monitor yourself while Auto Mode is working, since it builds, flashes and monitors the board itself. If it stops part way, the Req. panel offers to continue from the next category.

7-2. While it runs: set up Git

Open Git from the top bar. Setup creates a repository in the project folder, adds a .gitignore that keeps build output out of review, and makes one base commit.

Field	What to enter	Note
Remote repository	The URL of a repository you own (for example a personal test repository).	Added as `origin`. This is where your work is pushed.
Branch to work on	A short branch name. FirmGen prefixes it with FG-.	Every commit and push goes to this branch. The default branch is never committed to, and the name cannot be changed later.
Commit author	Your name and email.	Saved for this repository only.

7-3. While it runs: follow the work

- **Req. tab:** watch categories complete and the "requirements traced" count rise.
- **Firmware Topology:** once code appears, open it and find the BLE, MAC and logging parts of the design.
- **Files tab:** open a generated file and find where the FirmGen_XX name is built.
- **sources/ folder:** open the PRD, the requirements table and the criteria file produced in Steps 3 to 6.
- **Pipeline and Output:** see each phase (Planning, Coding, Building, Flashing, Monitoring) and read any build error FirmGen is fixing.

Step 8: Validate on the Board

When implementation completes, open **Monitor** (115200 baud). After a reset, the log should show, in order: the detected MAC address, the generated BLE name, and a message that advertising has started. Then check each PRD acceptance criterion:

PRD acceptance criterion	How to check it
The ESP32 boots and starts BLE advertising.	The advertising-started message appears after boot, with no errors or repeated reboots in the log.
The advertised name is FirmGen_ followed by the last 2 digits of the MAC address.	Compare the MAC printed on UART with the name printed on UART and the name your scanner shows.
The device is visible from a BLE scanner or mobile BLE app.	Scan with your phone. FirmGen_XX appears and stays visible (continuous advertising).

Read the evidence: a passing build and a "started" log are not the same as the device appearing in a scanner. The last criterion is only met when you see it on your phone.

Step 9: Generate Artifacts, Trace, and Push

1. (**Optional**)Open **Artifacts** in the top bar and run **Unit Tests**: FirmGen writes a test per requirement, then runs them. Find the test files and the results report.
2. (**Optional**)Run **Design Doc**: FirmGen refreshes the topology, then writes the design specification. Open it and find the BLE startup sequence.
3. (**Optional**)Open the traceability report in sources/. Follow the FirmGen_XX name requirement to its criteria, the code that implements it, and its test.

4. Review the diffs in the Git view and push to your branch.

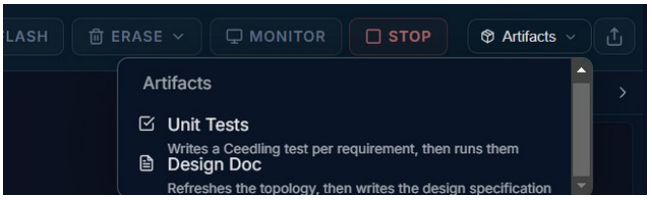


Figure 7. The Artifacts menu in the top bar exposes Unit Tests and Design Doc.

Two separate questions: traceability completeness and test pass status are different. A requirement can be linked to code while still waiting for test or on-board evidence. Unit tests check logic off the board; BLE visibility can only be confirmed on the device.

Continuing After the Lab

You have now run the full workflow once. There are two additional things you can use when you move on to your own hackathon project.

Iterating with direct prompts

Once a project exists, you can keep changing it by describing the change in the chat. FirmGen updates only the parts that need to change, then builds, flashes and monitors. Practise on your BLE project, for example by asking FirmGen to explain it:

Example prompt

```
Explain the current Firmware Topology, each module, and how the BLE name is built and advertised,
in simple language.
```

Prompt → Review (Topology, Agent Progress, Files) → Deploy (Build → Flash → Monitor) → Evidence → Modify + Redeploy

- **Prompt:** describe the next change and answer any questions FirmGen asks.
- **Review:** inspect the Firmware Topology, Agent Progress and changed files.
- **Deploy:** build, flash and monitor, as one operation or with the individual top bar actions.
- **Evidence:** read the per phase result and the runtime output.
- **Refine:** describe the smallest required change, redeploy, and observe.

Choosing a path for your own project

Use **Help me write one** or upload your own PRD, then pick the execution path: **Category-wise Manual Review** when you want to inspect each category, **Auto Mode** when the PRD is clear, or **Direct Prompt** for a fast first version. Durations are indicative and grow with project size, build iterations and hardware behavior.

Reference: FirmGen in Detail

Use this section during the hackathon to understand each capability and to work through problems on your own.

R1. Traceability and the generated repository

Requirement IDs are retained from the PRD through planning, design, implementation and testing. That lets you answer both review questions: "where was this requirement implemented?" and "which requirement caused this code or test to exist?"

Stage	Expected linkage
PRD → Requirement	Each requirement gets a stable ID and category. Explicit and inferred requirements stay distinguishable.
Requirement → Task	Tasks identify the requirement(s) they satisfy, so work can be reviewed before code is produced.
Requirement → Criteria	Criteria state how the requirement will be checked: host-testable, on-target, inspection-based, or needing instruments / HIL.
Requirement → Code	The generated files and relevant symbols / APIs that implement the requirement are recorded.
Requirement → Test	Test sources and reports carry the requirement ID (for example test_FR_001.c).
Code / Design → Requirement	Design sections, diagrams and source references point back to requirements for reverse navigation and change impact analysis.

Where a generated decision comes from an uploaded document, the workflow aims to keep the most specific reference available: datasheet page, section, register or table; schematic sheet, net or signal; coding guideline rule; or source file and symbol.

Repository layout

The output keeps specification inputs, target and toolchain assets, modular firmware, documentation and tests in predictable places, so FirmGen can change the right layer without mixing application logic with hardware-specific code.

Simplified folder structure (exact folders vary by project and platform)

```
project/
|-- sources/                  PRD, requirement categories and criteria, requirements table,
|                             traceability report (.md / .html / .xlsx)
|-- <generated workspace>/
    |-- <board folder>/       board and toolchain build files, vendor-generated code
    |-- application/          target/example application integration
    `-- firmware/
        |-- app/              startup orchestration
        |-- bsp/              board support
        |-- configs/          build, device and feature configuration
        |-- drivers/          portable device drivers (datasheet-generated drivers live here)
        |-- interfaces/       vendor-neutral contracts
        |-- modules/ services/  feature logic and application services
        |-- platforms/        platform-specific adapters
        `-- docs/             design spec and figures, Doxygen API docs, unit tests and results
```

Artifacts

Artifact	What FirmGen produces
Unit Tests	Ceedling / Unity-style unit tests mapped to the requirements, executed, with test files and an execution report stored in the project.

Artifact	What FirmGen produces
Design Doc	Refreshes the project topology and writes the firmware design specification: wiring, HLD, LLD, state and sequence flows, runtime task model, interface relationships and source references.
API documentation	Doxygen output for source-level API navigation.
Traceability report	Maps each requirement to acceptance criteria, implementation files and symbols, tests and verification status.

R2. Live Intelligence and target debugging

Live Intelligence is the entry point for runtime visibility. Debugging works through ST-Link or J-Link probes. Review any telemetry or debug changes FirmGen makes and verify them on the board.

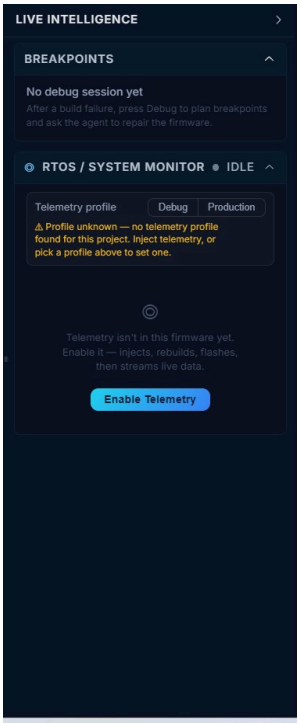


Figure 8. Live Intelligence: Breakpoints and the RTOS / System Monitor.

Area	What it does
Breakpoints	Target-side debugging entry point once a build / debug flow is available. After a build failure, Debug lets you plan breakpoints and ask the agent to repair the firmware.
RTOS / System Monitor	Runtime monitoring surface for telemetry-enabled firmware.
Telemetry profiles	Debug and Production profiles. "Profile unknown" means no telemetry profile was found for the project yet.
Enable Telemetry	Adds telemetry support when the firmware does not contain it: injects, rebuilds, flashes, then streams live data.

R3. Git

Git in FirmGen is intentionally minimal: visibility and basic project versioning, not a replacement for a full Git client. You can push and pull from the workspace, view commit history, and inspect file-level diffs (changed lines or the whole file) before reviewing or sharing changes.

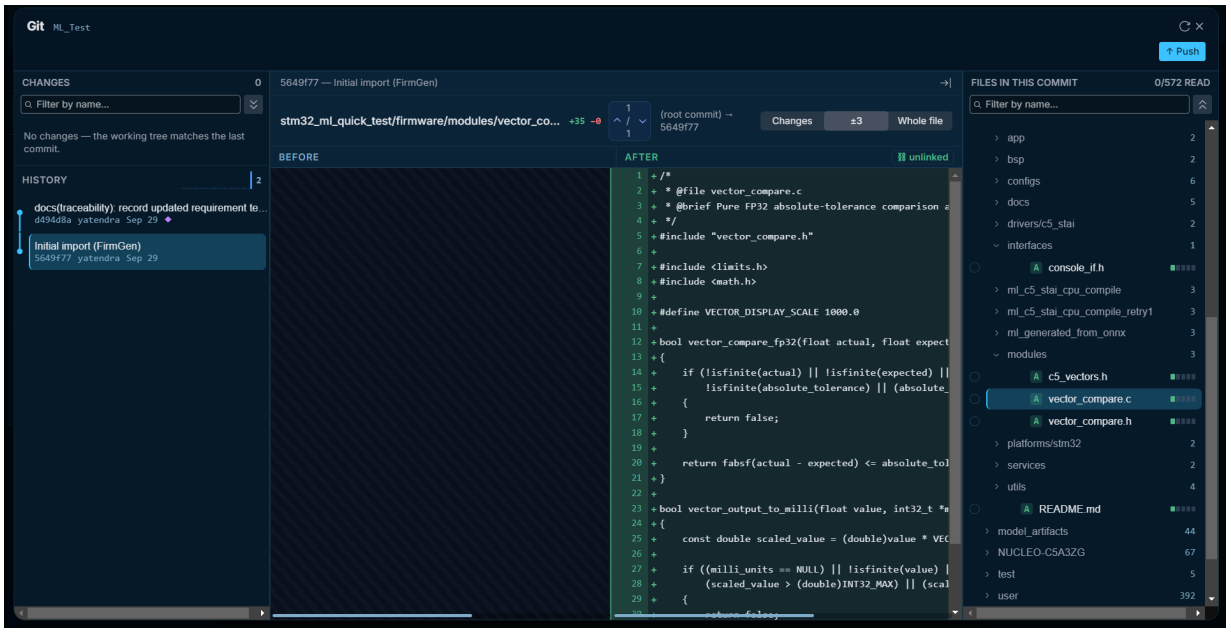


Figure 9. Git view with commit history, a file-level diff and the files in the selected commit.

R4. Hardware source references

For projects with sensors, external modules or add-on boards around the base board, FirmGen can use more than the base-board definition. Sensor datasheets and component documentation can be used as implementation references, add-on board information can be supplied alongside the base-board context, and reference material can justify pin mappings, register choices, interfaces and hardware-specific constraints.

R5. Coding guidelines

- Choose None, Defaults, BARR-C Embedded C Coding Standard, or Custom when you create the project.
- Custom guidelines can require API / function descriptions, and carry naming, modularity, comments, error handling and project conventions through generation.
- Custom guidelines can be used alongside standard-derived internal rules such as BARR-C or MISRA-inspired practices.

R6. Context-limit recovery

Long-running projects can build up enough chat history to approach the model context limit. When a chat crosses roughly 75% context and a context-window error appears, do not restart the project.

> **Recommended recovery:** open a new chat in the same project and instruct it to continue from the previous chat. The project files, requirements and artifacts are the stable project state; only the conversation moves to the new chat.

Quick Recovery Guide

Symptom	Check first	Useful FirmGen prompt or action
ESP32 build fails unexpectedly	That FirmGen is using ESP-IDF v5.5, not another installed version.	Confirm which ESP-IDF version this project is using.
Create project is disabled	Board selection, the Verify result, the project path and the toolchain entries.	Review the setup fields and tell me exactly what is preventing project creation.
Board or port is not detected	USB connection, cable type, driver, selected board, and whether Monitor is holding the port.	Diagnose device detection from the current state and propose the smallest checks in order.
Build fails	The first meaningful compiler or configuration error.	Fix only the first root cause build error, preserve the current architecture, and redeploy.
Flash fails	Correct board, port access, boot and reset state, and whether another tool is using the port.	Explain the flash failure and guide me through the minimum recovery sequence.
Deploy reports failure although the build passed	The runtime output. Success depends on device evidence.	Show me the runtime evidence from the last deploy and identify the first failure the device reported.
Requirements look bloated or wrong	How many are inferred, and whether they go beyond the PRD.	Reject or remove what you will not build; add what is missing; or improve the PRD and regenerate.
"N categories without tasks" warning	Whether you want to review plans before coding.	Use Task lists to plan them, or let Auto plan them during execution.
Implementation is still running near the end of the session	The Req. panel and pipeline: it is still making progress.	Let it finish, then do Steps 8 and 9. Do not restart it.
Flash fails while Auto Mode is running	Whether Monitor or another tool is holding the serial port.	Stop Monitor and let FirmGen retry.
Implementation stopped part way	The Req. panel category states.	Use the continue control in the Req. panel to resume from the next category.
Context-window error in a long chat	The context usage indicator.	Open a new chat in the same project and continue from the previous chat (R6).
FirmGen_XX does not appear in the BLE scanner	The UART log shows advertising started; phone Bluetooth is on; the scanner is not filtering by name or signal strength.	Show me the runtime evidence that BLE advertising started and check the advertising configuration against the PRD.
The XX digits do not match the address the scanner shows	ESP32 derives separate MAC addresses for different interfaces, so the address a scanner shows can differ from the MAC printed on UART.	Tell me which MAC address the firmware reads and which one BLE advertises with, and confirm the name follows the PRD.
A change modifies too much	Prompt scope and requested files; the Git diff.	Revert unrelated changes and apply the smallest change set needed for this requirement.
Firmware becomes unresponsive	Blocking delays, long callbacks, task starvation or repeated reconnect loops.	Identify blocking behavior and propose a bounded non blocking fix before editing code.

Symptom	Check first	Useful FirmGen prompt or action
Unit tests pass but the board misbehaves	Host tests do not cover electrical, timing or hardware faults.	Run on-target validation and read the monitor output, or use Live Intelligence.
Telemetry profile unknown in Live Intelligence	Whether the firmware contains telemetry yet.	Pick a profile or use Enable Telemetry.
Git branch name is wrong	The working branch name cannot be changed after setup.	Ask a facilitator before setting up Git again.
The agent appears to stall part way	The pipeline, Agent Progress, and the context usage indicator.	Tell me the last tool you ran and what you are waiting on.

Final Proficiency Self Check

Before working independently, confirm that you can do each of these without the guide:

- Install the prerequisites and confirm FirmGen works smoothly.
- Create a project with the correct platform, board and toolchain, and know what each knowledge input is for.
- Upload the BLE PRD, generate requirements, tell Explicit from Inferred, and freeze a reviewed set.
- Generate task lists and criteria, choose an execution path, and start or resume implementation.
- Point to the artifact each stage produced for the BLE project, and explain what the traceability report does and does not prove.
- Validate the firmware against the PRD acceptance criteria on the board and with a BLE scanner.
- Set up Git, inspect a diff and push to your branch.
- Recover from a context-window error without losing project state.

Summary

In the guided lab you set up FirmGen, took the BLE PRD from specification to reviewed requirements, task lists and acceptance criteria, implemented it on your ESP32, validated it against the PRD on the board and with a scanner, generated unit tests and a design document, traced a requirement end to end, and pushed your work. The same workflow carries through the hackathon: specify, review, generate, deploy, read the evidence, refine.

> **Hackathon mindset:** use FirmGen to accelerate engineering decisions, but keep reviewing requirements, architecture, build output, runtime behavior, memory headroom and error handling as the project evolves. A command that completes is not the same as firmware that works.

Appendix A: ESP32 BLE Broadcast PRD

This is the full text of the PRD used in this lab, provided at the event as **ESP32_BLE_FirmGen_Simple_PRD.pdf**.

ESP32 BLE Broadcast - Simple PRD

Objective

Create firmware for an ESP32 DevKit that starts BLE advertising automatically after boot.

BLE Device Name

The BLE advertised device name shall be generated dynamically in the format:

`FirmGen_XX`

XX shall be the last 2 hexadecimal digits of the ESP32 MAC address.

Example: If the MAC address is A4:CF:12:34:56:9B, the BLE name shall be FirmGen_9B.

Requirements

1. Initialize BLE after device boot.
2. Read the ESP32 MAC address at runtime.
3. Extract the last 2 hexadecimal digits of the MAC address.
4. Generate the BLE name as FirmGen_XX.
5. Start BLE advertising using the generated name.
6. Keep BLE advertising continuously.

UART Output

Use UART at 115200 baud for basic debug logs.

Print the detected MAC address.

Print the generated BLE device name.

Print a message when BLE advertising starts.

Acceptance Criteria

The ESP32 shall boot successfully and start BLE advertising.

The advertised name shall match FirmGen_ followed by the last 2 digits of the MAC address.

The device shall be visible from a BLE scanner or mobile BLE application.